

» Parte 2:

- Entidades: Jugadores, clubes, y transferencias:

} Relaciones:

- jugadores y transferencias: Transferencias guarda una id de cada jugador, esta id se usa para llamar de la colección al jugador específico que se nombra.

- clubes y transferencias: Transferencias guarda tanto el club de origen y el destino al que va, via id, para poder llamar los nombres de cada uno.

- No utilizamos embedding porque los jugadores y los clubes son entidades independientes de las transferencias. Al mantenerlos en colecciones separadas evitamos duplicar su información en cada transferencia y podemos reutilizar los mismos documentos mediante referencias (`_id`) cuando sea necesario

- Si lo usamos, debido a que usamos un elemento específico de una colección y dos de la otra, esto hace que solo lo llamemos una vez por cada uno, pq sino estaríamos haciendo una consulta de mas.

- Si se hubiera modelado de manera relacional, se ganaría una mayor integridad de los datos mediante claves foráneas y relaciones entre tablas. Sin embargo, se dependería de JOIN para obtener la información de jugadores, clubes y transferencias. En nuestro modelo NoSQL utilizamos referencias (`_id`) y `$lookup` solo cuando es necesario relacionar las colecciones, manteniendo una estructura más flexible.

} Esquema JSON:

```
{
  "jugadores": { "_id": 1, "nombre": "Lionel Messi", "edad": 36,
    "posicion": "Delantero", "nacionalidad": "Argentina", "clubId": 2
  },
```

```
  "clubes": { "_id": 1, "nombre": "Paris Saint-Germain", "pais":
    "Francia", "liga": "Ligue 1" },
```

```
  "transferencias": { "_id": 1, "jugadorId": 1, "IdclubOrigen": 1,
    "IdclubDestino": 2, "monto": 50000000 }
}
```

-----\$-----

» Parte 3: SQL Vs. No SQL:

SQL y No SQL son simplemente dos modelos de bases de datos distintos, los dos con sus características positivas y negativas, dependiendo de la naturaleza del proyecto a concretar.

> Estructura:

La principal diferencia se ve en que las bases de datos relacionales tienen una estructura para el almacenamiento de su información *rígida y predefinida,* basada en filas y columnas. Cosa que solo es una ventaja en proyectos donde, desde el inicio, se sabe con exactitud que tipo de información será almacenada, cuales serán sus relaciones y teniendo la certeza de que esta estructura no será alterada de forma significativa en el futuro. Por ejemplo, en sistemas financieros y de gestión de empresas.

En cambio, las bases de datos no relacionales ofrecen consistencia eventual, a cambio de consistencia estructural, basada en documentos y colecciones. Esto las hace más *flexibles* y permite al desarrollador ir adaptando la base a medida que el proyecto se desarrolla.

> Relaciones entre datos:

SQL administra las relaciones a partir del sistema de claves primarias y foráneas, especializándose en la ejecución de consultas complejas que involucran múltiples tablas. El modelo no relacional maneja los datos anidados y complejos dentro de documentos individuales.

> Escalabilidad:

El modelo SQL tiende a depender de la *escalabilidad vertical,* ya que la forma en que funciona dificulta el uso de múltiples servidores. Mientras que el modelo No-SQL tiende a tener una mejor escalabilidad en general, orientada a la horizontal.

> Lenguaje:

El modelo relacional utiliza estrictamente el lenguaje de consultas SQL, mientras que el no relacional hace uso de consultas basadas en JSON, con métodos como "find" "insert", "update", etc.

-----\$-----

» Parte 4:

```
// map → transformar
const nombres = usuarios.map(u => u.nombre);

// filter → filtrar
const baratos = productos.filter(p => p.precio < 100);

// reduce → acumular
const total = pedidos.reduce((suma, p) => suma + p.precio, 0);

// flat → aplanar arrays
```

```
const lista = [[1,2],[3,4]].flat(); // [1,2,3,4]

// flatMap → map + flat
const palabras = frases.flatMap(f => f.split(" "));
```

(Aclaración: Dentro del archivo "TransferMarker.js", también hacemos uso de bloques async/await, así como de bloques try/catch. No supimos incluirlo en el PDF sin repetir lo ya escrito, por lo tanto decidimos aclararlo de esta forma).

-----§-----

» Parte 6:

- El equipo usó IA como asistencia de redacción de texto y corrección de errores ortográficos. Asistencia en la sintaxis de diversos métodos, así como aclaración de dudas conceptuales menores (por ejemplo si el comando "use" debería ser incluido en el JS de la estructura de la base de datos a entregar).

- Asistentes de IA usados: Chat-GPT y GitHub Copilot.

-----§-----

» Parte 7:

- En general, apreciamos el enfoque práctico tomado en los parciales, ya que durante la cursada de Base de datos I, nos llamó la atención la falta de práctica con SQL. Modelo que no terminamos de entender hasta el choque de paradigma con lo no relacional.

- Sin ir más lejos, lo que más valoramos de lo que nos llevamos es el mejor entendimiento sobre bases de datos en general, el conocimiento de que existen diferentes modelos de las mismas y el manejo de nuevas herramientas como Mongosh y NodeJS.

- Además, apreciamos mucho la combinación del tema principal de la cursada (MongoDB y no SQL) con un lenguaje de programación como JavaScript, de forma que fue sencillo "ubicar" los conocimientos adquiridos con una base ya formada previamente en la carrera, a diferencia de lo ocurrido con Base de datos I donde, de nuevo, fue difícil "conectar" los temas vistos con lo visto en otras materias.